Amazon
MS
Goldman Sach
Meesho.net
Samsung

## Q Wildcard Matching

Given 2 strings.

$S_a$ → word {a-z}

$S_b$ → wild card {a-z, ?, *}

? → matches with any one character (exactly one)

* → can match with a sequence of character (including empty string)

Return true if the wild card string ($S_b$) matches with the word ($S_a$)

$S_a$ : a b x c l m
$S_b$ : a *
True

$S_a$ : a b x c l m
$S_b$ : a *
True

$S_a$ : a b c
$S_b$ : a ? _
False

$S_a$ : a b c d e f
$S_b$ : * ?

$S_a$ : a b c d e
$S_b$ : * *
True

$s_a$ : a b c  False

$s_b$ : a ? b * c

$s_a$ : ""  True

$s_b$ : * * * * * * * *

a * b

a c x y b ✓

a aa bbb asdfa dsfqsdb ✓

a b ✓

a aa cc asbd ✗

a ? b * c

a b c ✗

b c ✗

aa bczc ✓

ab rrt ✗

Word $s_a$ ($l_a$) : — — — — — —

Wildcard $s_b$ ($l_b$) : — — — —

→ [a-z]

→ ?

→ *

```
match (Sa, Sb, la, lb)
    if (Sb[lb-1] is [a-z])
        if (Sb[lb-1] == Sa[la-1])
            ret match (la-1, lb-1)
        else
            ret false
```

if ( $S_b[l_b-1]$ is '?' )

ret match( $l_a-1$, $l_b-1$ )

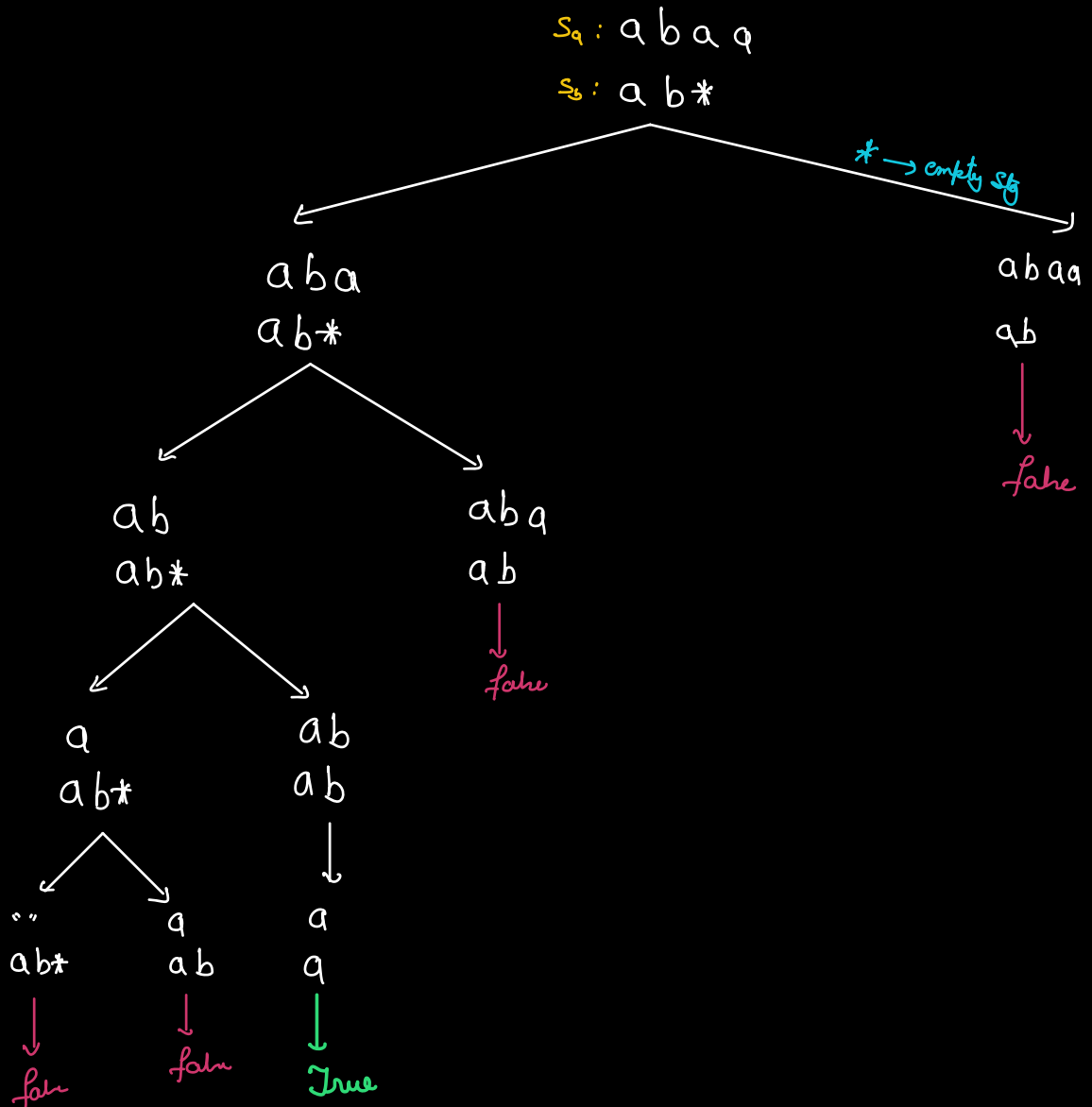
LCS

if ( $S_b[l_b-1]$ is '*' )

ret match( $l_a$, $l_b-1$ )

||

match( $l_a-1$, $l_b$ )



Base Conditions

```
if( la == 0 && lb == 0)
        ret true;
if ( lb == 0)
        ret false;

if ( la == 0){
        for( i=0; i < lb ; i++){
                if ( Sb[i] != "*")
                        ret false;
        }
        ret true;
}
```

Media.Net

Q Given 2 Strings A & B.
Count the unique ways in which we can get String B as a subSq of String A.

⇒ A : RABYBXBIT
B : RABBIT

RABBIT
RABBIT → 3
RABBIT

Case 1

$S_a[l_a-1] == S_b[l_b-1]$

RABBBB
RABB

Matching / Not matching

RABBBB
RABB
Count($l_a-1$, $l_b-1$)

RABBBB
RABB
Count($l_a-1$, $l_b$)

Case 2 $S_a[l_a-1] != S_b[l_b-1]$

RABBBIT
RABB

→

RABBBI
RABB

Base Case

A = "abc..."
B = ""

---

A = ""
B = "abc..."

```
int countSubSq(Sa, Sb, la, lb) {

    if(lb == 0)
        ret 1;
    if (la == 0)
        ret 0;
    if (Sa[la-1] != Sb[lb-1])
        ret countSubSq(Sa, Sb, la-1, lb);
    else {
        ret
            countSubSq(Sa, Sb, la-1, lb),
                +
            countSubSq(Sa, Sb, la-1, lb-1);
    }
```

- Memoization
- Iterative Solution

Q LPS (Longest palindromic subsequence)

Given a string. Return the length of LPS.

S : a g b d b a ⟶ a b d b a ⟶ 5

a a

b b

a g a

a b a

a d a

b d b

a b b a

a , b , g , a

a c b a c b

a → 2

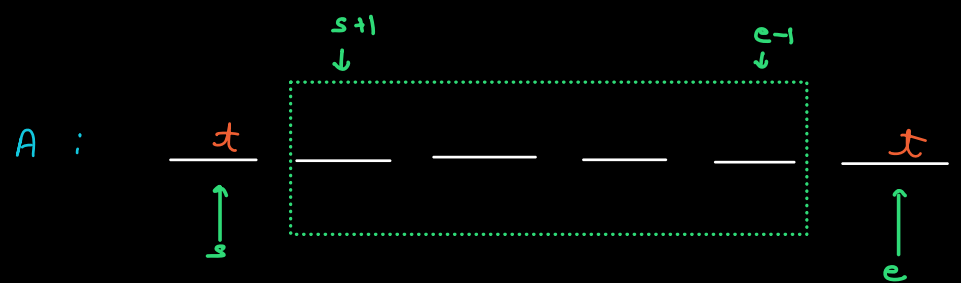
c → 2

b → 2

LCS ( a g b d b a ⟶ LPS

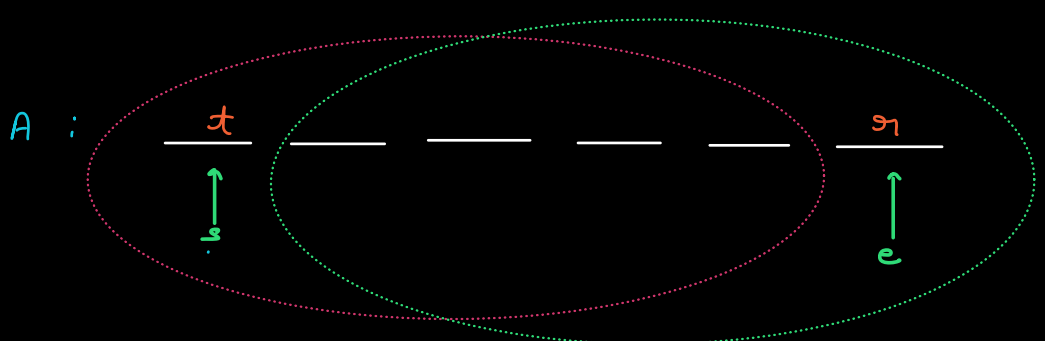
a b d b g a

Approach 1 ret LCS (s, Srev)




If (A[s] == A[e])

ret 2 + LPS (s+1, e-1)

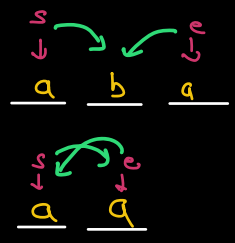


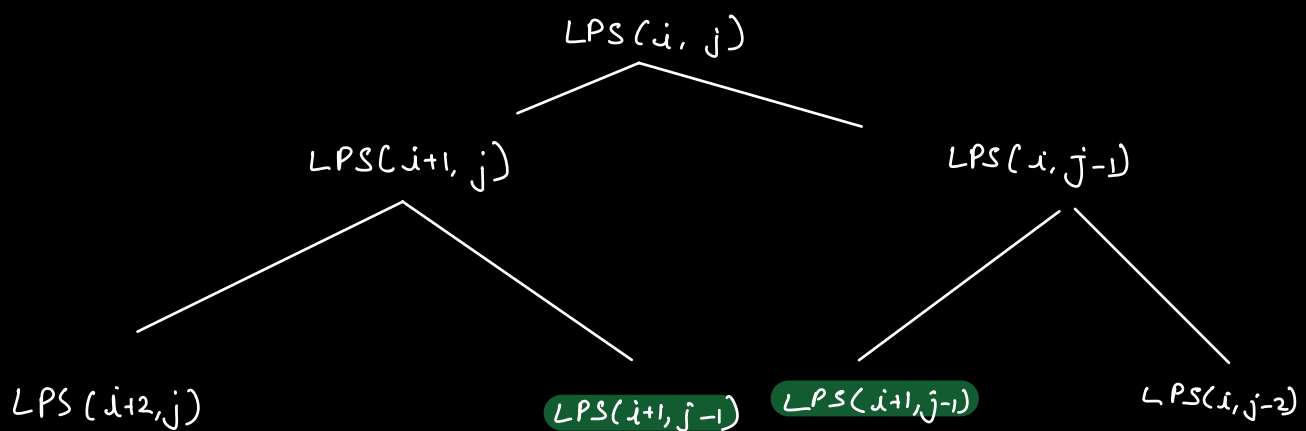

else ret max ( LPS (s, e-1),
LPS (s+1, e) )

```
int LPS( A, s, e){
    if (s==e) ret 1;
    if (s>e) ret 0;

    If (A[s] == A[e])
        ret 2 + LPS(A, s+1, e-1);

    else,
        ret max( LPS(A, s, e-1),
                 LPS(A, s+1, e) );
}
```




Total fn calls $\approx 2^N$

TC $= O(2^N)$

# unique fn calls $= O(N^2)$

LPS(A, s, e) → Returns the length of LPS in string A b/w the index s & e

HW : Implement bottom up solution